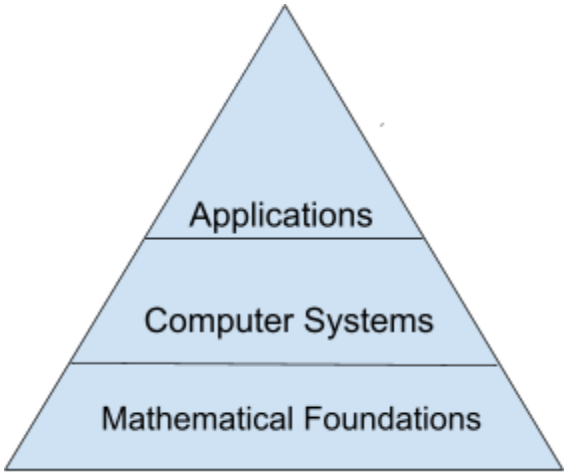
Computer Architecture Notes
An Overview of Hexagon DSP Processors
14-Nov-2024

By - Aditya Vikram Singh, EE22BTECH11001
Gagan Singla, EE22BTECH11021
Mayank Gupta, EE22BTECH11033
Sujal Gupta, EE22BTECH11052

Acknowledgements: We thank Ravishankar Kolachana and Sushim Shrivastava from Qualcomm Hyderabad for visiting us and conducting this lecture.

We all know how important proofreading is in the life of a computer scientist. All the applications are based on mathematical foundations.



Importance of DSP in Computer Science?

- The original need of DSP arised in the field of communication systems.
- But it found its application in the field of image processing and multimedia due to its power-saving ability.
- The problem with the CPU is that it works well for sequential processing and is extremely good at control processing but couldn't efficiently handle data processing.
- The GPU comes with good data processing capabilities but it lacks behind in control processing and is power extensive.
- DSP provides the best of two worlds and does both data processing and control processing in lesser power.
- For example, some phones that use DSP consume 50% less power than using CPU (for HDR cameras).

Hexagon DSPs in SnapDragon 820

The overview of the Snapdragon chipset is shown by Fig.1 (Link to image)

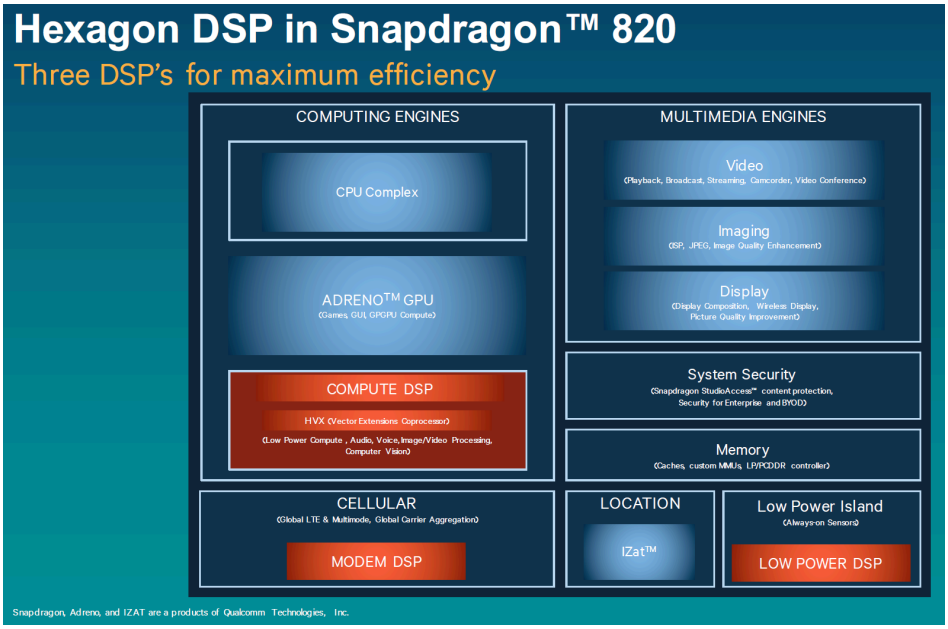


Fig. 1

The chip contains mainly three types of DSPs

- COMPUTE DSP - They are used for low-power computing operations and specialized tasks like audio/video processing and Computer Vision applications.
- MODEM DSP - Used for Global LTE and multimode communication (For, e.g. 2G, 3G, 4G)
- LOW POWER DSP - Used in sensors that constantly need to be active to detect any signal. (E.g., the Voice assistants need to be attentive to any wake-up voice commands)

Hexagon Vector eXtensions (HVX)

HVX is a SIMD (Single Input Multiple Output) architecture developed by Qualcomm for their DSP (Digital Signal Processor) cores.

Key Features of HVX

1. It is majorly used to compute significant workloads for advanced imaging and computer vision and other workloads found in mobile and embedded systems to be processed on the DSP instead of the CPU
2. HVX improves the capabilities of Hexagon DSP, enabling specialized processing like Content Adaptive Detail Enhancement.
3. It operates on wide vectors, enabling the processing of multiple data elements in parallel with a single instruction.

4. It achieves substantially better performance than the CPU in terms of power consumption.

HVX Architecture - SIMD Extensions

Large SIMD Extensions

1. HVX provides 1024-bit SIMD capabilities that help in parallel processing of large data.
2. Includes 4 vector-slot VLIW (Very Long Instruction Word) to execute multiple components simultaneously.
3. Achieves 4096 (1024*4) results in bits per cycle, showcasing high computational throughput for vector processing tasks.
4. It is Ideal for image processing and computer vision, where multiple elements or bits are processed in parallel.

Multiplications and Data Dimensions

1. Supports 256 8x8 multiplications or 64 16x16 multiplications in a single instruction.
2. 8x8 and 16x16 multiplications are optimized for low-resolution data and higher-precision calculations, respectively, It is very common in neural networks, image filtering, and matrix computations.

Vector Registers

1. Provides 32 1024-bit vector registers, which are storage units for handling large amounts of vector data.

No floating-point support

1. Designed to minimize power usage and area, floating point representation requires more power usage.
2. Supports 8-bit, 16-bit, and 32-bit fixed-point formats for processing integers.
3. Most of the ML workload doesn't require floating-point support extensively.

Specialized ISA

1. Includes instructions for sliding window filters, LUTs (Look-Up Tables), and histograms:
    a. Sliding window filters: Used for tasks like blurring or edge detection.
    b. LUTs: Accelerates operations like color correction or transformations.

    c. Histograms: Quantization and graph optimization are needed for these use cases, this architecture design is specific to the use case.
2. It is easier and faster for engineers since we have direct instructions available.
3. The growing 8K (7680 x 4320 pixels) resolution technology has increased the demand for efficient pixel-by-pixel processing, which HVX addresses with specialized, optimized instructions that significantly reduce time and power consumption.

Performance for Advanced Use Cases

1. Capable of handling UHD (Ultra High Definition) video post-processing: Processes high-resolution videos efficiently.
2. Supports 20 Mpixels camera burst mode processing: Handles rapid image captures from high-resolution sensors.

AI engineers often focus on training, but in reality, inference happens far more frequently, and optimizing for power efficiency in this domain is crucial, as training typically occurs only once.

HVX Architecture - Threading Model

The threading model makes use of the following units working together

- 4 Parallel Scalar Threads
  There are 4 Parallel Scalar Threads, each having a 4-way Very Long Instruction Word (VLIW), and have shared L1 and L2 caches. Each thread operates at a frequency of 500MHz, and the 4 scalar threads combined give a total scalar performance of 2GHz.
  An important point to note about the Scalar threads is that they are not particularly useful for vector calculations (and thus for Computer Vision and Machine Learning applications).
- 2 HVX contexts
  These 2 HVX contexts are controlled by any 2 of the 4 scalar threads. The remaining 2 scalar threads are used as a master controller for the remaining sequential and CPU-type functions. These HVX units avoid using the DDR memory controller and external memory for camera processing tasks and try to perform those tasks that fit within the existing L2 Cache size. This design aims to minimize latency, reduce battery consumption, and lower CPU and DDR access demands for camera and vision processing algorithms.

With Hexagon, the DSP's control code processor and HVX's computational code processor can operate simultaneously, enabling tight integration between control and computational tasks. This allows the DSP to use the result of an HVX computation to make a control code decision in the very next clock cycle. By streaming data directly

from the camera to the Hexagon DSP's local memory (L2 Cache), HVX enables advanced use cases extending beyond photography, including computer vision (CV), computational imaging, and machine learning.

HVX Architecture - Memory

The Hexagon processor features a unified byte-addressable memory. This memory has a single 32-bit virtual address space, which holds both instructions and data. Separate L1 instruction and data caches exist for program code and data. A unified L2 cache is partly or wholly configured as tightly coupled memory (TCM).

But, a large amount of data needs to be fetched for vectoring unit threads which a L1 cache can't store. To avoid unnecessary misses at each data fetch, vectoring type threads fetch data directly from the L2 cache which is bigger in size and is hence more suitable for our application. Whereas, for scalar units, fetching from L1 is more efficient as they require comparatively very less memory.

Hence, L2 is the first level of memory for vector units. Load/store operations that access memory through the L1 cache are referred to as cached accesses. Load/stores that bypass the L1 cache and directly access the L2 cache are referred to as uncached accesses.

Both L1 and L2 caches are Hardware Coherent. This means multiple processors or devices accessing shared memory have a consistent view of the data. It is a property of the hardware design and mechanisms that manage and synchronize data updates across different caches or memory units.
Simply speaking, when multiple processors have copies of the same data in their local caches, hardware coherence ensures that updates to the data are propagated correctly so that no stale or inconsistent values are used.

Prefetching is supported by both L1 and L2 caches. However, prefetching in L1 is **programmer-controlled** through explicit software instructions. Whereas, **Hardware-based prefetching** mechanisms in L2 attempt to predict future memory access patterns.

Vector units support the following types of load/store instructions:

1. **Unaligned Access** - Read or Write of data is not aligned to a specific number of bytes (8-byte, 16-byte or 32-byte etc.). This is sometimes required for efficient memory operations.

2. **Per-Byte Conditional Access** - Per-byte conditional access enables operations to act selectively on specific bytes of data within a vector, based on conditions applied at the byte level.

HVX Architecture - Programming

Programming of Hardware is a very important aspect to consider. Even if a very efficient hardware is designed, it is of no use if it can't be programmed to run properly. Virtual Memory - A layer of abstraction between the DSP's memory accesses and the physical memory. Memory addresses used by applications (virtual addresses) are translated to physical addresses using a **Memory Management Unit (MMU)**.

There are various programming models that are/were used in HVX DSP programming:

1. **DSP RTOS (DSP-based Real-Time Operating System)** - A multi-tasking, priority-based operating system that takes advantage of a DSP's high-performance features. This system is very compact as compared to our general purpose CPU.
2. **Halide** - It is a domain-specific programming language designed for high-performance image processing and computational photography. It provides an efficient and expressive way to write image processing pipelines.

System Features - Streaming Interface

1. Custom Camera Pre-Processing without DDR access saves power
2. Raw data from Image Sensor is streamed into L2 at up to 1.2 Gpixels/sec
3. Streamer formats / pads / aligns data for convenient vector processing
4. Data held in input & output circular buffers
5. HVX processed pixels can be streamed out to ISP HW

System Features - SMMU

1. The SMMU acts as a mediator between the DSP and the shared physical memory.
2. The SMMU maps virtual addresses used by the DSP to physical addresses in memory, enabling seamless data access without requiring manual address management.
3. This abstraction allows the DSP to handle large datasets and simplifies memory allocation for complex applications like image processing.
4. SMMU supports multiple Context Banks to allow sharing with multiple different address spaces on CPU.

System Features - Quality of Service
QoS (Quality of Service) refers to mechanisms and strategies designed to ensure that the performance of the DSP remains predictable and meets the requirements of high-priority workloads. QoS is critical to maintain performance under varying workload conditions. QoS ensures low-latency access to memory and data for time-critical operations on HVX.

L2 Partitioning - L2 cache can be soft-partitioned for different vector threads to avoid conflicts of data access which is necessary to maintain high QoS.

Internal QoS focuses on managing resources within the HVX or DSP subsystem itself. It deals with the allocation and prioritization of resources like processing units, memory access, and task scheduling. Software assigns priorities to threads.

External QoS focuses on managing resources and interactions between the HVX and other subsystems or components in a heterogeneous system. This includes the CPU, GPU, and system memory. System Level HW algorithms can throttle DSP traffic to ensure system performance.

Why is DSP more power efficient than CPU?

CPU	DSP
Out of Order / Superscalar	In Order / VLIW
Short Vector have high overhead	Long Vectors amortize overhead
L1 & L2 used for image data	Only L2 used for image data
Design Optimized for high MHz incurs high clock power overheads & high leakage	Design Optimized for low-power & lower MHz reduces overheads & leakage